

设计原则

一、面向对象设计的五大基石：SOLID 原则

SOLID 原则是指导类粒度职责分配、控制继承与访问耦合的核心武器。

1. SRP 单一职责原则 (Single Responsibility Principle / P13)

- 核心内涵：**一个类应该只有一个变化的原因。
- 实践指南：**如果一个类既负责业务数据又负责数据库持久化（如经典的 `Employee` 类混合了薪资计算和 `saveToDatabase`），它就拥有了多个变化原因，应当重构拆分为独立的领域对象和持久化仓库（Repository）。每个类应仅隐藏一个核心设计决策。

2. OCP 开闭原则 (Open-Closed Principle / P11)

- 核心内涵：**软件实体应当对扩展开放，对修改关闭。
- 实践指南：**当系统中引入新的业务类型时，应通过增加新的接口实现类或子类来扩展行为，而不是频繁去修改已有的 `if-else` 或 `switch-case` 分支。这常通过策略模式（Strategy）或多态调度来实现。

3. LSP 里氏替换原则 (Liskov Substitution Principle / P7)

- 核心内涵：**子类型必须能够替换其父类型而不改变程序的正确性。
- 契约视角的严格要求：**
 - 子类的前置条件必须满足：子类前置条件 $\leq$ 父类（即子类只能放宽前提，不能比父类更严格）。
 - 子类的后置条件必须满足：子类后置条件 $\geq$ 父类（即子类向调用方保证得更多）。
 - 子类必须无条件维护父类的所有**不变式**（Invariants）。
- 典型反例：**几何学上的“正方形继承长方形”会破坏长方形宽高独立的契约不变式；“企鹅继承鸟类”并在 `fly()` 方法中抛出 `UnsupportedOperationException`，强制强化了前置条件，直接违反了 LSP。通常需通过接口分离（如引入 `Flyable` 接口）进行重构。

4. ISP 接口隔离原则 (Interface Segregation Principle / P6)

- 核心内涵：**不应该强迫客户端依赖它们不使用的方法。
- 实践指南：**严格防范包含太多无关方法的“肥接口”。应当将臃肿的大接口细化拆分为聚焦的职责接口（如将 `Worker` 拆分为 `Workable` 和 `Feedable`），让调用方精准依赖。

5. DIP 依赖倒置原则 (Dependency Inversion Principle / P12)

- 核心内涵：**高层模块不应依赖低层模块，两者都应依赖抽象；抽象不应依赖细节，细节应依赖抽象。
- 实践指南：**在代码编写时坚决践行“面向接口编程，而不是面向实现编程”（Program to an interface, not an implementation）。高层业务逻辑（如 `BusinessLogic`）不应直接持有 `MySQLDatabase` 等具体低层实现类，而应持有抽象的 `Database` 接口，通过依赖注入（DI）在运行时装配细节。

二、 模块化控制与信息隐藏原则 (P1 - P5, P8 - P10)

这组原则主要源自 David Parnas 的思想，强调将“容易改变的设计决策”作为秘密锁在模块内部，对外只暴露稳定的接口，实现真正的黑盒复用。

1. 基础控制原则 (P1 - P4)

- **P1 全局变量有害**：全局变量会穿透所有模块的防线，是信息隐藏的直接反义词，应坚决避免。
- **P2 优先显式表达**：利用清晰的接口契约、显式命名和文档表达约束，而不是依赖开发人员之间隐含的口头约定。
- **P3 不要重复 (DRY - Don't Repeat Yourself)**：同一个系统决策或业务规则绝对不应在多处编码，重复意味着未来变更时需要多处同步修改。
- **P4 面向接口编程**：将访问耦合从复杂的成员变量级别降为仅依赖接口，是降低模块间依赖强度的通用手段。

2. P5 迪米特法则 (Law of Demeter / 最少知识原则)

- **核心内涵**：只与你直接的朋友交谈。一个方法应当只能调用自身 (`this`)、方法参数对象、该方法内部新创建的对象以及自身持有的直接数组成员/字段。
- **防范消息链问题**：坚决避免火车残骸式 (Train Wreck) 的连续链式调用 (如 `a.getB().getC().doSomething()`)。这种做法暴露了多层间接对象的内部结构；应重构为在直接朋友中封装行为的“委托”模式 (如 `a.delegateToB()`)。

3. P8 组合优于继承 (Favor Composition Over Inheritance)

- **继承 (白盒复用) 的缺陷**：子类被迫暴露于父类的内部实现细节中，继承耦合极深，父类的轻微改动极易破坏子类的正确性。
- **组合 (黑盒复用) 的优势**：仅通过稳定的抽象接口相互依赖，不仅职责清晰，还允许在运行时动态地配置和切换不同的底层策略。

4. 封装的精细化落地 (P9 - P10)

- **P9 为变更而封装 (Encapsulate What Varies)**：准确识别系统中频繁变化的需求点 (如算法策略、对象创建、数据格式等)，将其隔离为独立模块。类的职责应表现为状态 (State, 次要秘密/实现变更) 与行为能力 (Behavioral Capability, 对外承诺的接口) 的有机结合。

💡 **封装的五种类型 (A-E)**： * **类型 A (ADT)**：隐藏内部数据表示 (如使用访问器替代公开字段)。 * **类型 B (内部结构)**：隐藏数据的组织结构 (如使用 `Iterator` 遍历而不暴露容器类型)。 * **类型 C (其他对象引用)**：不直接返回或泄漏类持有的内部对象引用。 * **类型 D (类型信息)**：隐藏运行时的具体类型，依赖多态调度。 * **类型 E (潜在变更)**：使用策略模式等手段封装可能会频繁变化的策略或算法。

- **P10 权限最小化**：严格限制类和成员的访问级别 (`private` → 包私有 → `protected` → `public`)。从最严格的 `private` 开始，只有确实需要时才逐步放宽，因为对外的每个 `public` 都是很难撤回的承诺。

三、 方法级设计原则：SOFA 原则 (P14)

与 SOLID 这种针对于“类与构件粒度”的原则互补，在编写具体的方法函数时，必须遵循 **SOFA 原则**：

- **S —— Short (方法要短)**：方法体通常建议控制在 **20-30 行**以内，过长的方法应拆分为具名清晰的辅助私有方法。
- **O —— One thing (只做一件事)**：方法名应当能用一个简单的短句完整描述。如果在口头描述该行为时需要用到“and”，说明它承担了多重职责，应进行拆分。
- **F —— Few arguments (参数要少)**：参数越多，调用方需要知道的内部细节就越多，访问耦合暴增。当参数超过 3 个时，应当考虑引入“参数对象”进行封装。
- **A —— Abstraction level consistency (抽象层次一致)**：高层业务逻辑中绝对不能混入底层实现细节。例如在处理订单的高层方法里，不能直接硬编码写死一条具体的 SQL 执行语句，而应将其封装为 `persistOrder(o)` 方法，保持整段代码在同一业务抽象层面上运作。

四、GRASP 通用职责分配软件模式

GRASP 并不是具体的设计模式（如 GoF 23 种模式），而是指导我们**如何把系统的宏观职责分派到具体的类与对象身上**的底层基础原理。

GRASP 核心模式	解决的核心问题与分配规则	带来的核心演化设计效益
Information Expert (信息专家)	将职责分配给 拥有完成该职责所需全部必要信息 的那个类。也就是说，“ 谁持有数据，谁就负责基于该数据的操作 ”。	能够自然地形成职责委托链，防止数据和实现细节外泄，维持类的高内聚。
Creator (创建者)	当满足以下条件时，由类 <i>B</i> 负责创建类 <i>A</i> 的实例： <i>B</i> 组合/聚合了 <i>A</i>；<i>B</i> 记录 <i>A</i> 的实例；<i>B</i> 紧密使用 <i>A</i> 或持有创建 <i>A</i> 所需的初始化数据 。	让类自己负责创建密切相关的生命周期对象，避免引入额外的外部工厂类，保持低耦合。
Controller (控制器)	专门引入一个控制器类（非 UI 组件）来负责接收和协调系统级交互事件。	将表现层（UI）与核心领域层 业务解耦 ，防止业务逻辑向展示层泄漏，确保两层独立演化。
Low Coupling (低耦合)	属于评估类设计方案的元原则。在分配职责时，需仔细权衡哪种选择 引入的外部依赖强度最低 。 优先依赖抽象接口	确保某类的变动对外部的影响面被降到最低， 类更易复用和独立理解 。
High Cohesion (高内聚)	同样是元原则。 确保一个类承担的职责都是紧密相关且聚焦的 ，绝不拼凑无关的操作。	类的职责高度清晰，易于管理和调试， 维护时改动范围小 。

决定方法的归属

决定谁来 new 对象

分离前后端

信息隐藏

与控制器模式相关

2. 控制策略简述

- **集中式控制 (Centralized)**: 一个“上帝对象”控制所有的流程和决策，其他对象只作为单纯的数据载体。导致主类极度臃肿。
- **分散式控制 (Dispersed)**: 控制逻辑散布在各个对象中，没有明确的中心，对象之间互相调用。容易形成网状耦合，难以追踪流程。
- **委托式控制 (Delegated)**: 控制器对象管理整体流程，但将具体的计算、校验等职责**委托 (Delegate) **给下属的专业对象（如验证器、策略类）完成。是最符合面向对象“高内聚低耦合”原则的策略。

3. 排课情境的选择

- **推荐使用：委托式控制策略。**
- **原因**: 排课系统业务极度复杂，如果让 `ScheduleService` 既负责读写数据，又负责判断时间、教室、教师等各种冲突，会导致该 `Service` 变成难以维护的“上帝类”（违反单一职责原则）。使用委托式，`ScheduleService` 只负责掌控宏观流程，将具体的“教师冲突检查”、“教室冲突检查”委托给各自的 `ConflictStrategy` 类去处理，将数据持久化委托给 `Repository`，使得系统职责清晰、扩展性极强。